Chronicle: durable session memory for Claude Code

Abstract

Stock Claude Code is excellent at searching the repository in front of it. Chronicle targets the missing substrate: prior agent sessions. It is a hook delivered memory layer that retrieves project context from the user's own session JSONLs and renders a `<chronicle-context>` block into the next turn before the model has to know it exists. On prior session dependent coding tasks, the current evidence shows Chronicle improving completion rate, prior context signal, and rediscovery overhead against stock Claude Code, with cost and wall time results that are directionally favourable but not yet fully powered. The conversation time pilot, where memory is injected mid task during repository exploration, is closest to the strong claim and is the most consistent across all four endpoints (hard pass, signal, cost per success, search per success). This write up describes the architecture, the eval harness, what the clean paired data does and does not yet support, and the eval pack required to earn a stronger public claim.

Keywords: coding agents; memory; retrieval; evaluation; Claude Code; prior session context.

1 The framing this write up earns, and the one it does not

The most useful framing for Chronicle is the narrowest one. Stock Claude Code can search the repo in front of it. It cannot search what already happened: the design decisions, the rejected alternatives, the project internal names, the corrections, and the partially completed work that only exist in earlier sessions. Chronicle's bet is that closing that one gap is worth a per prompt retrieval layer.

The defensible claim this write up supports is:

> *On tasks where the necessary information exists in prior session history but not in the current repo or prompt, Chronicle measurably improves completion rate, prior context signal, and rediscovery overhead. Cost and wall time results are directionally favourable, with the strongest cost adjusted gains on the conversation track where memory is injected during the agent's own search behaviour.*

The bolder framing, "Chronicle is faster and cheaper than stock Claude Code on every coding task," is not yet earned, and arguably should never be the public claim. A strong general purpose coding agent on a task that does not need prior session context should already do well; expecting Chronicle to beat it on a self contained refactor is the wrong test. The `chronicle_advantage_pack` section at the end specifies the eval shape that would justify the bolder version. Until that pack runs, the write up sticks to the defensible version above.

The launch style version of the same idea, pre paid by the eval pack, is:

> *Stock Claude Code searches your repo. Chronicle searches what already happened. On prior session dependent tasks, that translates into more completed tasks per dollar, with less repository search.*

2 Why Chronicle exists

I started digging into `MEMORY.md` after spending a day switching between two Claude Code accounts, re explaining the same deployment process in every new chat. What I found was a single markdown file capped at 200 lines or 25 KB, loaded at session start, written to by a prompt deciding what was worth remembering, and periodically pruned by something called "Auto Dream." My current project's file had one line. The other projects I had been working on were the same story. The cap is not silly, since pulling everything into every prompt does not scale. But the result is that, for me, prior session knowledge essentially did not exist between turns. The interesting question is whether a per prompt retrieval layer over the actual session JSONLs can do better than a single curated markdown file, while keeping latency low enough that hooks remain practical.

That question is what Chronicle is trying to answer. I had built RAG before and written parsers before, so the cost of trying was contained.

3 How Chronicle works

Chronicle is a per prompt retrieval layer for Claude Code. It indexes the project's real session JSONLs (the transcripts CC writes to `~/.claude/projects/<encoded>/<uuid>.jsonl`) and writes a `<chronicle-context>` block into the model's view of the next turn. The substrate is the user's own prior sessions; nothing synthetic.

3.1 Two retrieval substrates

Chronicle runs two indexes in parallel.

Substrate A: conversation and action chunks. The session JSONLs are split into roughly 400 word windows and embedded with `BAAI/bge-small-en-v1.5` (33M params, 384-d, around 6 ms encode steady state), then stored in sqlite plus sqlite-vec. The spoken chunk path keeps user and assistant exchange and deliberately excludes `tool_use` markers, `tool_result` content, `thinking` blocks, slash command echoes, and synthetic tool result as user turns. The current chunker also creates `behavioral` chunks for standard Bash, Edit, Write, and MultiEdit tool inputs and `tool_result` chunks for filtered failures or narrow deploy and publish success evidence. Chunk IDs are content addressed (`sha256(session || first_turn || content)[:16]`), so unchanged JSONLs re index for free.

The reasoning: a query about `make_cache_key` should match the discussion of the function, not a code dump that happens to contain it.

Substrate B: typed claim memory store. Markdown files extracted offline from the same JSONLs by Claude Haiku. Each file has metadata at the top (`kind`, `polarity`, `referent`, `source_session`, `source_turn`) and is indexed twice: FTS5 (BM25) for keyword and sqlite-vec for embedding, combined with Reciprocal Rank Fusion. A graph of replaced memories sits alongside, with links between old and new claims, plus cycle checks at construction time.

3.2 The execution loop

A `UserPromptSubmit` event with both substrates enabled currently runs through this shape:

1. `chronicle hook` reads the payload from stdin (prompt, session_id, project_dir, recent_turns, compaction_count).
2. A path triage classifier (`SKIP / LIGHT / FULL`) inspects prompt shape. SKIP exits silently; LIGHT only adds session state blocks.
3. Conversation search tries the daemon first. If the daemon is unavailable, the hook falls back to an in process embedder and store.
4. On the non composer conversation path, the index is refreshed by JSONL `mtime`. On the composer path, Chronicle searches an existing conversation index; incremental refresh there is still a limitation.
5. The query runs through dense, hybrid, or multi query search depending on config and prompt shape, with `min_top_cosine`, rerank, and MMR controls when enabled.
6. Candidate chunks are re ranked with polarity, topic overlap, recency, list answer penalties, stale and replacement signals, and citation use scores.
7. The typed claim store is searched alongside the raw chunks; results use kind, polarity, supersession, temporal, and lexical scorers.
8. A routing weights classifier (sub millisecond, no LLM) outputs source and memory kind weights based on prompt patterns.
9. The composer allocates source budgets, removes duplicate overlap, formats blocks, and writes a `<chronicle-context>` block to stdout.
10. Claude Code prepends the block to the model's view of the turn.

The hook always exits 0; failures inside `run_hook` are caught and logged. The 5 second timeout in `settings.json` is the safety net. Steady state hot path is tens of milliseconds once the embedder is warm.

3.3 Hook events

The default installer wires only `UserPromptSubmit`. The full set is opt in via `~/.claude/settings.json`:

Event	Purpose
`UserPromptSubmit`	Front of turn injection for the changing per prompt path
`SessionStart`	Stable prelude from cached conventions and session state
`PreToolUse:Read/Grep/Glob`	Mid turn context injection for read and search tool results
`PreToolUse:Bash`	Action time guard; normalises and classifies, checks against learned conventions
`PostToolUse:Read`	Annotates memory file reads with replacement graph status
`Stop / SubagentStop`	Citation grading on what the agent actually used
`SessionEnd`	Typed claim extraction over the just finished session

Table 1: Hook events Chronicle can wire into `settings.json`.

3.4 Triage and the composer

The triage module is two related classifiers. Path triage (`SKIP / LIGHT / FULL`) decides whether retrieval runs at all. Routing weights decides how the substrates are blended. The composer uses these weights to split the context budget, sort results from each source, remove overlap (a chunk that is just a memory's source turn gets dropped under `prefer_memory`), and format block sections. It does not yet do one global weighted merge because memory scores and chunk scores live on different scales.

3.5 Supersession

The supersession graph is built from each memory's `revises` field. It exposes `is_superseded`, `superseded_by`, `latest_in_chain`, and a bi temporal `validity_at(t)` projection inspired by Zep and Graphiti. The graph powers two retrieval behaviours: a `SUPERSEDED_PENALTY = 0.5` multiplicative downrank during scoring, and the `PostToolUse:Read` annotation that catches mid turn reads of stale memory files even when they bypass the front of turn injection.

4 Why this isn't standard RAG

The framing that frontier labs are moving away from RAG is too broad to be useful for understanding what Chronicle is doing. Two trigger patterns need to be distinguished. Tool based memory, which includes MCP style memory tools and agent driven retrieval, depends on the model recognising that prior context exists and then choosing to query for it. That is the failure mode MEMTRACK [2] identifies: agents do not reliably call memory tools, and adding memory tools can increase redundant tool calls without improving recall. Naive RAG avoids that specific failure because retrieval fires on every user query rather than on the model's discretion. The MEMTRACK objection lands on agentic memory tools, not on RAG broadly.

Chronicle's trigger pattern is RAG shaped, not tool shaped. A `UserPromptSubmit` hook runs on every turn and queries the session history substrate before the model has to know anything is missing. Where Chronicle differs from generic RAG

is in the substrate and the ranking. Standard RAG indexes an external corpus and ranks primarily by dense similarity. Chronicle indexes the user's own prior session JSONLs and the typed claims extracted from them, then ranks with polarity, recency, list answer penalties, citation use scores, and the supersession graph layered on top of dense similarity. The narrower limits of dense retrieval still apply. Embeddings struggle with negation, time, and composition; "we use Postgres" versus "we used to use Postgres" is exactly the kind of distinction a memory layer most needs to make and exactly what cosine similarity is worst at. The supersession graph addresses that case directly, downranking superseded claims when both the old and the new make it into the candidate set. Retrieval before the model has reasoned about the turn is also still an early guess, so Chronicle can miss, and a miss looks the same as a true absence to the rest of the system.

What Chronicle's design changes is not who triggers retrieval, since that is automatic in both naive RAG and Chronicle. It is what is being retrieved over and how it is ranked. For a small, user owned session history where the relevant context is reachable but invisible to the model by default, indexing the right substrate and ranking with session aware scorers is the structural shift that matters. The hook versus tool distinction is what separates Chronicle from MCP style memory, not from RAG itself.

5 Design decisions

A few decisions carry most of the design.

Hook delivery, not MCP. Hooks put memory into every prompt without asking the model to opt in. MCP would make memory a tool the model chooses to call, which adds a latency round trip and runs into the MEMTRACK failure case described above where agents do not reliably call memory tools.

LLM free hot path. Per prompt work runs locally: embedding, sqlite-vec lookup, heuristic re ranking, triage, and composition. `ANTHROPIC_API_KEY` is only needed for offline typed claim extraction and optional graders or planners.

BGE-small over MiniLM. A tradeoff, not a leaderboard pick. BGE-small worked better on Chronicle's own session data: short project specific decisions where the query and source chunk often do not use the same words. One representative case was `polarity-fold-into-kind-match`: BGE returned the chunk containing `POLARITY_NONMATCH_FLOOR = 0.4`, while MiniLM put it outside the top 20. The cost is carrying a local sentence-transformers model at all, especially on cold start, so the daemon matters.

Separating spoken text from tool output. File dumps and command output can drown out the discussion around them. The current chunker keeps spoken chunks clean, then indexes tool inputs and selected tool results as separate chunk kinds. That keeps code and output evidence available without letting it dominate every conversation window.

Content addressed IDs. `chunk_id = sha256(session || first_turn || content)[:16]`; memory IDs use the same shape. Unchanged JSONLs can be skipped, and cache artifacts stay portable across eval workdirs.

6 What we are trying to measure

There is not yet a public benchmark that directly measures Chronicle's shape: a coding agent using raw prior work sessions as memory while completing later tasks. MEMTRACK [2] has the right shape but its main finding is the problem Chronicle tries to bypass. SWE-Bench [3] and its variants test single task software engineering, not retrieval over prior sessions. LongMemEval [6] and LOCOMO [4] are multi session but conversational. None of them measures what Chronicle measures: whether retrieval can recover useful project context from raw, unstructured prior session traces where most of the material is irrelevant to the next task.

So the eval harness is custom. It is not meant to prove that the benchmark design is ideal; the evaluation design is part of what needs to improve. False positives, weak checks, convergent tasks, and unfair scenarios are all useful findings.

6.1 Co primary endpoints, instead of one headline metric

A previous version of this write up reported each track's strongest single metric, which made it harder to see the through line. The shape of the claim Chronicle should be tested on has three co primary endpoints, plus a Chronicle specific diagnostic:

Endpoint	Question
Hard pass	Did the task get done at all?
Cost per hard pass	Among completed tasks, what did each completion cost?
Wall time per hard pass	Among completed tasks, how long did each completion take?
Repo search per hard pass	How much rediscovery did the agent do on the way to a completion?

Table 2: Co primary endpoints.

Signal pass, "did the agent visibly use prior context," sits next to these as the Chronicle specific quality check. Equal hard pass with higher signal means Chronicle changed *what* gets done, not *whether* it gets done. That distinction matters because most retrieval experiments only measure completion.

6.2 Tracks

6.3 Variants

The two arm `chronicle-on` versus `baseline` comparison is the intended headline. Headroom is a useful but secondary arm: a separate memory tool with the opposite delivery style, distilled into rules rather than per prompt lossy but fresh, with

Track	Hook wired	Question
controlled	UserPromptSubmit	Does retrieval help on a tightly scoped, single source session decision?
dogfood	UserPromptSubmit	Does retrieval help on real, noisy, multi session Chronicle dev history?
conversation	PreToolUse:Read/Grep/Glob	Does mid turn retrieval change read and search behaviour?
vague	UserPromptSubmit	Does retrieval help on open ended user prompts mined from real sessions?
behavior	UserPromptSubmit	Does retrieval help the agent follow project specific rules and conventions mined from sessions?
integration	UserPromptSubmit	Does retrieval help on multi step refactors that compose decisions across many prior sessions?

Table 3: Eval tracks and the question each is meant to answer.

Variant	Setup
chronicle-on	Hook wired in .claude/settings.json; substrate copied into per tuple workdir; conversation index pre warmed
baseline	Stock claude -p with empty settings.json; no hook, no substrate
headroom-on	Comparison arm: Headroom's offline learn --apply writes CLAUDE.md + MEMORY.md and runs an HTTP proxy on 127.0.0.1:8787

Table 4: Variants compared in the harness.

a public repository[1] that has 1k+ stars.

6.4 Fairness controls

The harness tries to keep Chronicle wins tied to retrieval rather than leaked answers. Signal checks should require a concrete fact from prior sessions: a project internal name, an exact value, a rejected alternative, or a multi piece recipe. The starter repo and prompt should not hand the answer to the baseline. Each scenario is tested against ideal/ and noop/, and long source sessions can use source_cutoff_timestamp so later eval writing discussion does not leak into the substrate. Each tuple runs inside a sandboxed Docker container with a per tuple repository copy, isolated home and config paths, and taint checks for suspicious reads or writes. Building that isolation took non trivial time, but it was necessary to make the comparison credible.

7 Headline results

Confidence intervals throughout this section are bootstrap intervals over paired comparisons. Power diagnostics use a paired McNemar style approximation to estimate how many paired tuples would be needed to detect a 15 percentage point effect; those diagnostics size the next benchmark and should not be read as achieved power claims for the current pilots. Pairs with a taint_kinds flag on either arm are excluded before analysis.

The headline results follow the natural progression from the simplest task shapes through to the focused conversation pilot. The vague track tests open ended user prompts where the hard pass bar is intentionally low. Behaviour tests project specific rules and conventions on the largest paired cohort. Integration composes decisions across many sessions in multi step refactors. The conversation time pilot closes the section as the most controlled test in the dataset and serves as the proof of concept for the next stage of the project: the chronicle_advantage_pack benchmark designed to prove that Chronicle is the most efficient option on prior session dependent work.

7.1 Vague track: open ended prompts and answer quality

The vague track is the simplest task shape in the dataset: open ended user prompts mined from real sessions, with a deliberately low hard pass bar that only verifies the agent wrote a non trivial answer.md. That makes hard pass a completion metric rather than a quality metric, so the headline result for this track lives in manually graded answer quality. The weak hard pass condition is a known current limitation of the vague track and is being addressed in the next release, where a stricter quality based hard pass criterion will replace the file existence check.

After taint filtering, the data contains 62 paired seed comparisons over 24 scenarios.

Unit	Chronicle wins	Ties	Manual Chronicle score
Scenario level	14	10	0.792; bootstrap 95% CI [0.688, 0.896]
Seed pair level	36	26	0.790; bootstrap 95% CI [0.726, 0.847]

Table 5: Vague track manual quality grading.

Chronicle produced the more useful answer in 36 of 62 paired seed comparisons and tied in the remaining 26. The baseline did not win any clean pair. The cost sanity check is more modest: across 60 paired completed comparisons, mean paired cost was +$0.0073 with bootstrap 95% CI [-$0.0642, +$0.0947], median paired cost was -$0.0471, and mean paired wall time was +15.0 s with CI [-2.1 s, +33.5 s]. Vague track cost is not where the win lives. Quality is. The cost adjusted story shows up in the conversation pilot and in the dominance and signal per dollar tables below.

[1] https://github.com/chopratejas/headroom

7.2 Behaviour track

Behaviour is the largest powered track. It tests whether retrieval helps the agent follow project specific rules and conventions mined from prior sessions, all delivered through the `UserPromptSubmit` front of turn surface. After taint exclusion, the dataset has 99 baseline, 100 Chronicle, and 102 Headroom tuples.

Variant	Tuples	Hard pass	Signal pass	Cost N	Mean cost	Wall N	Mean wall
Baseline	99	92/99 (92.9%)	51/99 (51.5%)	92	$0.0709	92	55.7 s
Chronicle	100	100/100 (100%)	81/100 (81.0%)	100	$0.0866	100	69.3 s
Headroom	102	96/102 (94.1%)	60/102 (58.8%)	96	$0.0760	96	72.0 s

Table 6: Behaviour track per arm summary. Cost N and Wall N are the number of tuples that completed and contributed to mean cost and mean wall time.

Comparison	Metric	N paired	Mean delta	Median delta	Bootstrap 95% CI
Chronicle vs Baseline	Hard pass	98	+0.061	0.000	[+0.020, +0.112]
Chronicle vs Baseline	Signal	98	+0.296	0.000	[+0.204, +0.388]
Chronicle vs Baseline	Completed task cost	92	+$0.0121	+$0.0014	[-$0.0029, +$0.0302]
Chronicle vs Baseline	Completed task wall time	92	+14.1 s	+9.3 s	[+7.1 s, +22.3 s]
Chronicle vs Headroom	Hard pass	100	+0.060	0.000	[+0.020, +0.110]
Chronicle vs Headroom	Signal	100	+0.230	0.000	[+0.130, +0.330]
Chronicle vs Headroom	Completed task cost	94	+$0.0112	-$0.0040	[-$0.0029, +$0.0281]
Chronicle vs Headroom	Completed task wall time	94	-0.9 s	-18.2 s	[-12.1 s, +10.7 s]

Table 7: Behaviour track paired comparisons.

Behaviour is the strongest correctness and signal result. Hard pass lift over baseline is +6.1 points, CI [+2.0, +11.2]; signal lift is +29.6 points, CI [+20.4, +38.8]. Against Headroom, Chronicle is also ahead on both completion and signal. The cost penalty is not resolved by this run because the paired completed task cost intervals cross zero; wall time against baseline is slower. The observed discordance power diagnostic puts Chronicle versus baseline signal at 98 current pairs versus roughly 111 required for a 15 point effect under the observed discordance rate, so signal is close to but slightly under target. Hard pass is above target.

7.3 Integration track

Integration is smaller (12 untainted tuples per arm), so the confidence intervals are much wider. It is reported here as directional evidence rather than a powered result.

Variant	Tuples	Hard pass	Signal pass	Cost N	Mean cost	Wall N	Mean wall
Baseline	12	10/12 (83.3%)	4/12 (33.3%)	10	$0.5393	10	269.0 s
Chronicle	12	12/12 (100%)	9/12 (75.0%)	12	$0.4832	12	263.4 s
Headroom	12	9/12 (75.0%)	6/12 (50.0%)	9	$0.5276	9	247.9 s

Table 8: Integration track per arm summary.

Chronicle's completion and signal means are higher; the medians and means line up the right way; but the paired CIs are wide because there are only 12 seed pairs. Cost is indistinguishable from zero in all pairwise integration comparisons. The power diagnostic puts Chronicle versus baseline signal at roughly 320 paired tuples for a 15 point effect under the observed discordance rate. Treat integration as supporting evidence, not a primary claim.

7.4 Conversation pilot: proof of concept for the next stage

The focused Sonnet pilot is the most controlled test in the dataset. It uses four conversation time scenarios, three seeds per scenario, and two variants: `baseline` and `chronicle-on`. It is intentionally small: a pilot designed to test whether the conversation time delivery surface is worth scaling up to a fully powered benchmark, not the final powered benchmark itself.

All 24 tuples completed with zero hook silent failures, zero errors, and empty persisted `taint_kinds`. Chronicle hard passed every focused pilot tuple and doubled the number of full signal passes. Cost per full hard pass was 53% lower for Chronicle and wall time per full hard pass was 18% lower. Because the cohort is only four scenarios, the bootstrap intervals remain wide: Chronicle's scenario level fractional hard pass lift was +0.250 with 95% CI [0.000, 0.500], and its fractional signal lift was +0.375 with CI [-0.083, 0.833]. This is therefore strong pilot evidence, not final proof.

A stronger earlier Sonnet pilot run on the same scenarios produced consistent results: baseline at 12/18 hard and 8/18 signal versus Chronicle at 18/18 hard and 18/18 signal, with zero taints and zero hook silent failures. I treat that earlier run as continuity evidence, not a separate powered result.

The clean scenario level story is:

- `live-context-cache-policy-refinement`: the current warmup says to use the production policy, but the exact `ttl_seconds=947` value only exists in prior substrate. Baseline missed it in all three seeds; Chronicle passed all three.
- `presearch-known-file-route`: decoy importer routes make broad repo search misleading. Chronicle found the remembered route in all three seeds; baseline fully hard passed two seeds but failed the full signal check in all three.
- `live-context-quiet-negative-control`: both arms passed; Chronicle did not leak unrelated remembered context into a simple local typo fix.
- `live-context-accepted-webhook-correction`: Chronicle hard passed every seed.

Comparison	Metric	N paired	Mean delta	Median delta	Bootstrap 95% CI
Chronicle vs Baseline	Hard pass	12	+0.167	0.000	[0.000, +0.417]
Chronicle vs Baseline	Signal	12	+0.417	+1.000	[-0.083, +0.833]
Chronicle vs Baseline	Completed task cost	10	-$0.0051	-$0.0242	[-$0.1257, +$0.1388]
Chronicle vs Baseline	Completed task wall time	10	+17.3 s	-8.4 s	[-39.1 s, +77.8 s]

Table 9: Integration track paired comparisons.

Variant	Tuples	Full hard	Full signal	Frac. hard	Frac. signal	Cost / hard	Time / hard
Baseline	12	8/12	5/12	0.750	0.542	$0.0560	81.0 s
Chronicle	12	12/12	10/12	1.000	0.917	$0.0266	66.5 s

Table 10: Focused conversation pilot.

The conversation pilot is where Chronicle should matter most, and it does. It is also where the cost story lines up cleanly with the completion story: Chronicle wins on all four co primary endpoints. That makes this pilot the proof of concept for the next stage of the project, the `chronicle_advantage_pack` benchmark, where a larger and more rigorous test set is designed specifically to prove that Chronicle is the most efficient option on prior session dependent coding work. The next benchmark should be larger and should separate three conditions: Chronicle on, Chronicle off with normal in session context, and Chronicle off with session restarts between turns, so that the memory contribution can be disentangled from the in session context contribution.

8 Pairwise dominance, signal per dollar, and search displacement

Track level paired tables answer the academic question of whether Chronicle won this metric on average. They tend to bury the practical question: in how many seed pairs did Chronicle make the agent both better *and* cheaper, in how many did it cost more for nothing, and how often did it actually lose. Three additional cuts make that practical picture legible.

8.1 Pairwise dominance buckets

For every paired seed comparison, classify the outcome on quality (signal pass) and on completed task cost. "Cheaper" and "costlier" mean the Chronicle arm had lower or higher cost on the same seed pair where both arms completed; "unpaired cost" means only one arm completed and a paired cost delta cannot be computed.

Track	Pairs	W+ch	W+co	W+up	T+ch	T+co	T+up	L+ch	L+co	L+up
behavior	98	13	15	6	24	40	0	0	0	0
conversation (conv-8)	24	1	1	5	4	13	0	0	0	0
conversation (focused)	12	3	0	4	0	5	0	0	0	0
integration	12	2	4	2	4	0	0	0	0	0
vague	62	24	10	2	13	13	0	0	0	0

Table 11: Pairwise dominance buckets. W = win, T = tie, L = loss; ch = cheaper, co = costlier, up = unpaired cost.

Two things in this table matter more than the totals. First, no track has any clean "Chronicle loss" cell: no (loss + cheaper), no (loss + costlier), no (loss + unpaired). Across 208 clean paired comparisons covering five tracks, Chronicle did not produce a quality regression. Second, the dominant cells for Chronicle wins are on the cheaper side in vague (24 vs 10) and conversation focused (3 vs 0); for behaviour, win+cheaper and win+costlier are similar (13 vs 15). The per track shape predicts cleanly which tracks survive a "more tasks per dollar" test today and which require the `chronicle_advantage_pack`.

8.2 Signal per dollar

Signal per dollar, defined as successful Chronicle relevant signal passes divided by total cost, captures the lazy "does Chronicle just spend more" question more honestly than mean cost.

Read as cost per signal pass, Chronicle is cheaper than baseline on every track in this run: 22% on behaviour, 26% on the broader conversation expansion, 64% on the focused conversation pilot, 54% on integration, 41% on vague. Vague signal pass uses the harness signal definition rather than the manual quality grade, so the manual quality result should still be cited separately.

The focused conversation number, where Chronicle delivers 31.3 signal passes per dollar against baseline's 11.2, is the strongest single number in the dataset. It is also a pilot, so it should carry pilot caveats whenever it is quoted publicly.

8.3 Search displacement

Search displacement is the most Chronicle specific metric in the conversation track: did memory point the agent at the right place before broad repo discovery? It is computed as the count of `Read`, `Grep`, and `Glob` tool calls per tuple.

Average reductions are 25 to 36% of baseline search depth, and average turn deltas are -1.71 to -2.00. The biggest reductions concentrate in `presearch-known-file-route`, where memory pointed Chronicle straight at the right route while baseline broadly searched the decoy importer files:

This is the cleanest mechanism story Chronicle has so far: the conversation hook injects the right route, the agent stops searching for it, the run is shorter and cheaper. The gain transfers across both pilot runs without scenario tuning.

Track	Variant	Tuples	Hard	Signal	Total cost	Signal / $	$ / signal
behavior	baseline	99	92	51	$6.9780	7.3	$0.1368
behavior	chronicle-on	100	100	81	$8.6565	9.4	$0.1069
behavior	headroom-on	102	96	60	$7.5079	8.0	$0.1251
conversation (conv-8)	baseline	24	19	12	$1.5133	7.9	$0.1261
conversation (conv-8)	chronicle-on	24	24	18	$1.6820	10.7	$0.0934
conversation (focused)	baseline	12	8	5	$0.4480	11.2	$0.0896
conversation (focused)	chronicle-on	12	12	10	$0.3192	31.3	$0.0319
integration	baseline	12	10	4	$5.6266	0.7	$1.4067
integration	chronicle-on	12	12	9	$5.7988	1.6	$0.6443
integration	headroom-on	12	9	6	$4.9529	1.2	$0.8255
vague	baseline	62	60	17	$22.9064	0.7	$1.3474
vague	chronicle-on	62	62	33	$26.0480	1.3	$0.7893

Table 12: Signal per dollar across tracks and variants.

Run	Pairs	Chronicle search	Baseline search	Mean delta	% lower	Turn delta
conversation (conv-8)	24	3.42	4.54	-1.12	38%	-1.71
conversation (focused)	12	2.83	4.42	-1.58	33%	-2.00

Table 13: Search displacement on the two conversation runs.

9 Trace stories

Two scenarios from the integration track illustrate the qualitative Chronicle and baseline gap. *Because the test data came from my own code sessions, I have slightly tweaked the naming of the prompts and the logic flow after the fact. The actual evals were run with the real data.*

Integration replacement status, seed 0. The prompt asked what happened to a partially completed replacement of an older third party integration with a newer provider. Both runs were clean and both passed the hard completion check. Baseline cost $0.2812; Chronicle cost $0.1359. Baseline answered from the current repository: it concluded that the newer provider had been mostly wired up, that dead code from the older provider remained, and that a current API error was blocking ship. That is a plausible repo only answer, but it misses the actual historical question of what happened to the replacement work. Chronicle recovered the prior session fact that the replacement edits had only existed in an uncommitted working tree and were later overwritten by a reset style restore of one backend function file. The only surviving record was a diff captured in the earlier conversation, so the work needed to be reconstructed rather than merely found in the current repo.

Document link sync status, seed 0. The prompt asked whether a daily CRM sync populated links to company documents. Both runs were clean and both passed the hard completion check. Baseline cost $0.1371; Chronicle cost $0.1163. Baseline searched the current codebase and concluded that document links did not exist anywhere in the repo. Chronicle answered the current code question but added the missing historical context: a prior document sync job had been the planned source of truth, matching cloud drive files to companies, storing matched links in a dedicated table, and patching the CRM field; a later decision had the daily sync read manually set links back from the CRM and store synthetic link rows keyed to the CRM record. The baseline saw absence; Chronicle explained both the absence and the intended design.

9.1 Bad news pairs

These are the pairs most likely to embarrass a naive Chronicle claim. They are useful failures rather than mysterious noise: each one points at a fixable surface.

Two failure modes account for most of these. `fp-os-blocker-until-secrets` is the "retrieval injects advice but does not enforce it" failure: the agent completed the task without verifying or acknowledging the secrets blocker. The fix is to promote mined blockers and proscriptions into the `PreToolUse:Bash` guard so active prerequisites become `warn`, `ask`, or `deny` controls instead of passive context. `feature-from-pattern` is the procedural memory failure: the agent completed the task without satisfying the prior component pattern check. The fix is behavioural and tool result indexing plus stable procedural slots, broadly Headroom's aggregate by frequency advantage. The `pre-warm-and-port-unit-tests` rows are tasks where a hard pass only completion mask hides a quality difference and Chronicle pays for extra exploration; those scenarios should be reshaped to reward the project specific recipe rather than just file production.

10 What the data does not yet support

There are five claims this run does not yet earn, and being explicit about them is part of what makes the defensible framing honest.

1. **"Chronicle is cheaper per completion on every track."** The vague track paired cost CI crosses zero. The behaviour track paired cost CI crosses zero. Only the conversation pilot shows a clean cost per completion win, and it is a 12 tuple pilot. The signal per dollar tables show a more consistent picture, but signal per dollar and cost per completion are different statistics.
2. **"Chronicle is faster per completion overall."** Behaviour wall time is slower against baseline (paired CI [+7.1 s, +22.3 s]). Conversation pilot wall time is faster. Vague wall time CI crosses zero. Wall time in this run is task shape dependent.
3. **"Chronicle beats Headroom on cost or wall time."** Chronicle has higher signal lift than Headroom and equal or better hard pass, but the paired cost and wall time CIs against Headroom mostly cross zero on the behaviour and integration tracks.

Run	Scenario	Seed	Search delta	Chr. vs base	Signal delta	Cost delta
focused	presearch-known-file-route	2	-8	2/10	+1.0	-$0.0632
conv-8	presearch-known-file-route	0	-7	2/9	+1.0	-$0.0462
conv-8	presearch-known-file-route	2	-7	2/9	+1.0	-$0.0507
focused	presearch-known-file-route	0	-6	2/8	+1.0	-$0.0451
focused	presearch-known-file-route	1	-6	2/8	+1.0	-$0.0723
conv-8	presearch-known-file-route	1	-6	2/8	+1.0	-$0.0433

Table 14: Top search displacement seed pairs.

Track	Scenario	Seed	Quality Δ	Signal Δ	Cost Δ	Wall Δ	Search Δ
behavior	fp-os-blocker-until-secrets	0	+0.00	-1.00	+$0.5667	+256.3 s	-3
conversation (conv-8)	live-context-accepted-webhook-correction	2	+0.00	-1.00	+$0.0327	+29.0 s	+1
conversation (focused)	live-context-accepted-webhook-correction	0	+0.00	-1.00	+$0.0265	+38.0 s	+1
conversation (focused)	live-context-accepted-webhook-correction	1	+0.00	-1.00	+$0.0123	+28.5 s	+2
integration	feature-from-pattern	1	+0.00	-1.00	-$0.0240	-34.5 s	-2
integration	feature-from-pattern	2	+0.00	-1.00	-$0.1288	-36.6 s	-2
integration	feature-from-pattern	0	+0.00	-1.00	-$0.2743	-78.4 s	+2
vague	pre-warm-and-port-unit-tests	1	+0.00	+0.00	+$1.7838	+295.3 s	+16
vague	pre-warm-and-port-unit-tests	2	+0.00	+0.00	+$0.8602	+234.4 s	-2

Table 15: Bad news pairs.

4. **"Chronicle helps on tasks that don't depend on prior session context."** That is the negative control claim and Chronicle should not be expected to hold it. The `chronicle_advantage_pack` should explicitly include negative controls.
5. **"Chronicle is powered for signal lift."** Behaviour signal is at 98 paired tuples versus a roughly 111 tuple target for a 15 point effect. Integration signal would need roughly 320 paired tuples. The current results are pilot evidence.

A stronger public framing requires that the next benchmark close at least the first three of these gaps, with the third probably moved to a separate Chronicle versus Headroom matrix.

11 The path to a stronger claim: chronicle_advantage_pack

The current evidence supports the defensible framing. Earning the bolder framing, "more completed tasks per dollar with less search," requires a benchmark whose scenarios are designed around the prior session dependence assumption from the start, rather than assembled from existing tracks.

That benchmark is `chronicle_advantage_pack`. The shape:

- 20 scenarios by 3 seeds by 2 variants for the minimum public pilot (120 tuples), 40 scenarios for the stronger version (240 tuples). Headroom is added as a second matrix only after baseline versus Chronicle is clean.
- Scenario mix: 10 exact path or file route cases, 8 correction or refinement cases, 8 risky procedure or blocker cases, 6 multi step recipe cases, 4 integration status cases, 4 negative controls.
- Calibration requirements: `ideal/` passes hard and signal; `noop/` fails the memory dependent signal; baseline cannot pass from local repo uniqueness alone; expected retrieval hits are recorded in `scenario.yaml`; source cutoff prevents eval authoring leakage.
- Co primary endpoints, in this order: hard pass, cost per hard pass, wall time per hard pass. Signal pass, signal per dollar, repo search calls per successful task, turns per successful task, source present to rendered to obeyed waterfall, and taint, hook, and native memory side effects sit alongside as secondary endpoints.
- Headline figure: budget dominance curve. The x axis is cost or wall time budget, the y axis is tasks solved. Claim shape: for any practical budget, Chronicle solves more tasks. This figure is the public facing version of the pairwise dominance table above.

Four product changes should land before the big pack runs:

1. Make rendered Chronicle blocks more action shaped when the current prompt is operational. This is a generic composer rule keyed on prompt class and retrieved evidence type, not a vendor token patch. The goal is to convert "Chronicle knew but agent ignored it" misses into hard passes.
2. Add a repo search metric directly from the Claude Code stream log, separate from native Claude memory reads, so the search displacement story is reproducible across tracks rather than only conversation.
3. Make native Claude memory side effects a first class aggregation field. `live-context-accepted-webhook-correction` was useful for catching this confound and the metric needs a permanent home.
4. Tighten pre search hook latency with the daemon path so conversation track gains are not erased by cold embedder load. The daemon exists; the background mining sweep needs hardening before the daemon is the only path.

12 Current limitations

Chronicle has only just started, but it has reached basic operational ability: it can index real sessions, inject context, run evals, and produce useful run logs and metrics. The remaining work is about making that reliable enough to use without babysitting it.

Implementation.

- The `compose.enabled` path does not yet refresh JSONLs incrementally.

- `USER_AUTHORED_BOOST` is defined but not wired into scoring. Citation prior scores exist, but writeback is opt in and needs more validation.
- Pinned policy can be shown early on `UserPromptSubmit`, but it is not yet a full `PreToolUse` hard block.
- Action mining works for conventions, rituals, and things not to do, but broader detection coverage is still partial.

Architecture.

- Dense retrieval can still mix up negation, time, and replaced claims.
- Pre turn retrieval can miss because the model has not reasoned about the task yet.
- Several keyword lists assume English and common HTTP and POSIX error wording.
- The SessionStart cache benefit is plausible but not measured yet.
- Large project cost comes mainly from typed claim LLM extraction, not local chunk indexing.

Eval harness.

- The runner defaults to three variants, while the headline comparison is usually two arm.
- Calibration coverage is incomplete.
- Prewarm fingerprint omits `source_cutoff_timestamp`, so cutoff changes can reuse stale or leaky caches.
- Hook logs and WIN, TIE, and LOSE classification need stricter correctness gates.
- The taint scanner does not cover every host transcript leak path yet.

The repo is not yet open source clean; the package shape is currently a research monorepo rather than a clean public package.

13 Further developments

Build out the current use case. The near term goal is to make Chronicle reliable as a day to day Claude Code memory layer. Make `compose.enabled` the default route by refreshing the conversation index there, connect user authored claims to ranking, turn pinned rules into `PreToolUse` blocks, keep improving action mining, measure whether the SessionStart prelude reduces cost, generalise English specific keyword lists, and clean up the package shape. The other immediate issue is cold start: BGE-small is fast once loaded, but sentence-transformers cold load can cost several seconds, and repeated hook fires turn that into visible delay. The daemon path exists but is not yet reliable as a cache producer; the honest near term goal is to use it first for latency and harden it as a producer second.

Measuring memory over large coding systems. The next step is a system that measures memory recall across large, ongoing codebases: real session streams, task N evaluated against sessions 1..N-1, and checks that ask whether the agent used the remembered project context. The pattern Chronicle's own pilots already show, that selected and well filtered prior context helps while unfiltered or wrongly selected context can hurt, is a generalisable hypothesis worth testing at scale rather than a finding that one pilot can settle. Contained single task evals are useful, but they are not the best model for where agentic programming seems to be heading: longer projects, recurring agents, and memory that carries across work.

Petri style memory audits. Anthropic's Petri framework [1] is an alignment auditing tool, not a coding memory benchmark, but its harness shape is what is relevant here. A Chronicle version would borrow that shape in two layers: a scenario generator seeding each scenario with the memory challenge under test (stale memories, missing prior decisions, misleading near matches, retrieved chunks the agent *should* ignore), and a Petri style auditor driving the multi turn coding interaction against a target agent with Chronicle on or off. The judge would score not just final correctness but whether the agent relied on the right memory, ignored bad memory, or wasted turns re discovering context that should have been shown. That complements the current scripted check harness rather than replacing it.

Citation faithfulness. One useful distinction in recent RAG citation work is citation correctness versus citation faithfulness: a cited source can support a claim even if the model did not rely on it. Wallat et al. [5] report up to 57% of citations lack faithfulness in standard RAG QA. Chronicle can test a smaller, practical version of that question in coding by joining the injection log, the citation log, file edits, and Bash calls to ask whether the agent's action actually depended on the memory that was injected and cited. If unfaithful citations predict failed checks, citation faithfulness becomes a useful signal before a run finishes rather than an after the fact paper metric.

References

[1] Anthropic. Petri: An open-source auditing tool to accelerate ai safety research. `https://www.anthropic.com/research/petri-open-source-auditing`, 2025. Accessed 2026-05-04.

[2] Darshan Deshpande, Varun Gangal, Hersh Mehta, Anand Kannappan, Rebecca Qian, and Peng Wang. MEMTRACK: Evaluating long-term memory and state tracking in multi-platform dynamic agent environments. In *NeurIPS 2025 Workshop on Scaling Environments for Agents (SEA)*, 2025. URL `https://arxiv.org/abs/2510.01353`.

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL `https://openreview.net/forum?id=VTF8yNQM66`.

[4] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, Bangkok, Thailand, 2024. Association for Computational Linguistics. URL `https://aclanthology.org/2024.acl-long.747/`.

[5] Jonas Wallat, Maria Heuss, Maarten de Rijke, and Avishek Anand. Correctness is not faithfulness in retrieval augmented generation attributions. In *Proceedings of the 2025 International ACM SIGIR Conference on Innovative Concepts and Theories in Information Retrieval (ICTIR)*, pages 22–32. Association for Computing Machinery, 2025. doi: 10.1145/3731120.3744592.

[6] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL `https://openreview.net/forum?id=pZiyCaVuti`.